

OpsDesk: storing data safely in production

Which tools we use, what they cost, and why - in plain language

The short version

- **Keep PostgreSQL.** It is the right database for tickets, CMDB, SLA and audit, and it scales to billions of rows. What needs to change is *how* we run it: a standby server, continuous backups, a connection pool and monthly partitions for the big tables.
- **Replace MinIO.** The free MinIO edition we use for attachments is no longer maintained. SeaweedFS (free) or Ceph (free, if a storage team runs it) are the replacements. They speak the same S3 protocol, so the app needs no code change.
- **Keep monitoring data out of Postgres.** Hardware metrics, logs and heartbeats go to Zabbix, Prometheus and Loki. Today heartbeats land in the audit table: about 55 million rows a year for 5 sites and 500 devices.
- **Everything can run on free, open-source software.** Paid options only buy back staff time (managed database, cloud storage, hosted error tracking). None are required.
- **Fix the go-live blockers first** (page 6): rate limits, upload size, Redis recovery and access checks.

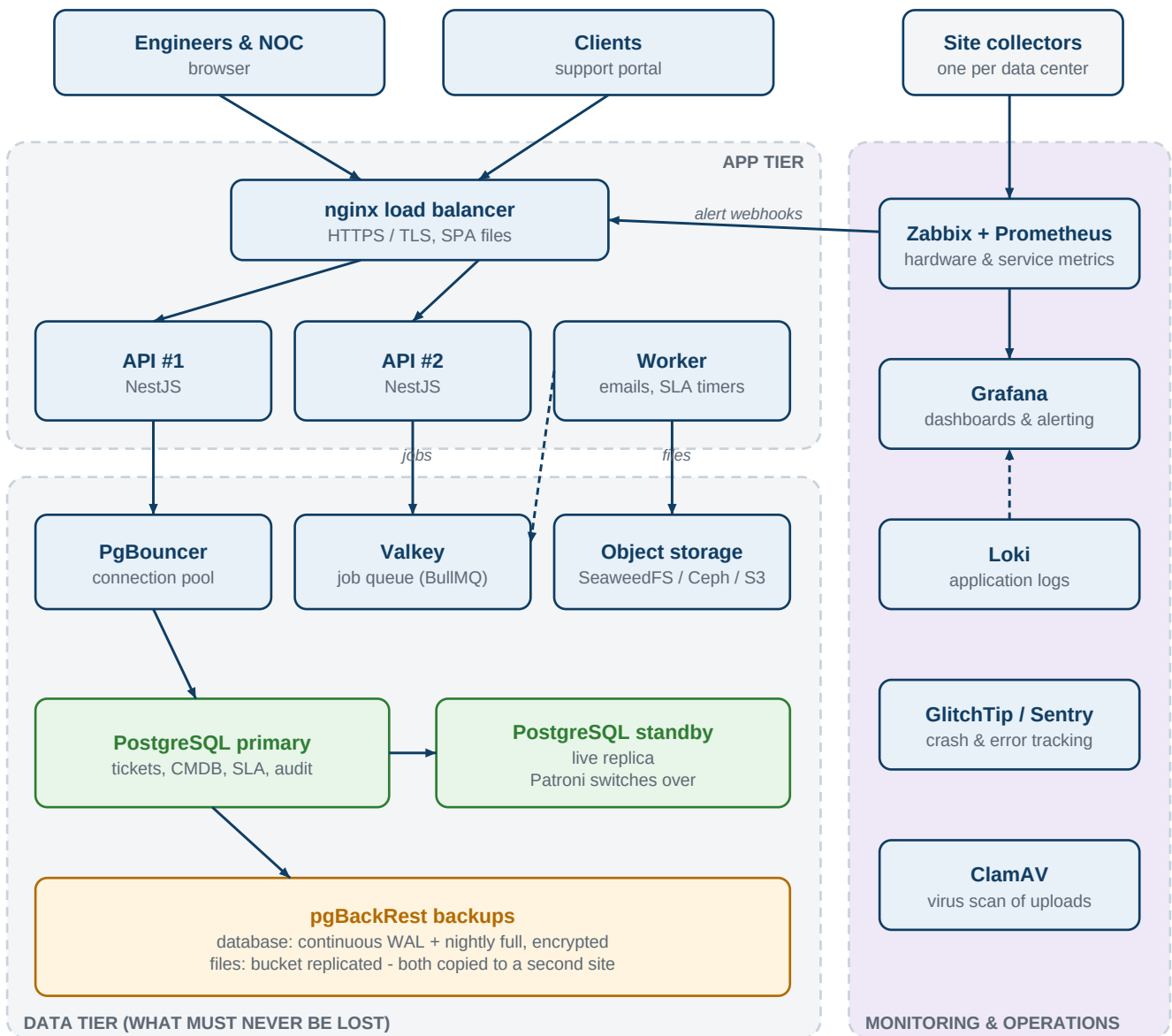
FREE open source, no licence fee	PAID subscription or pay-as-you-go	FREE + paid option : free to self-host, paid if hosted for us
---	---	---

A note on licences

Some tools here (Zabbix, Grafana, Loki) use the **AGPL** licence. For a company using them internally, unmodified, it costs nothing and requires nothing. It only matters if we modify the tool's own code and offer it to outsiders as a service. Our own OpsDesk code is not affected by the tools it talks to.

Diagram 1 - The production setup

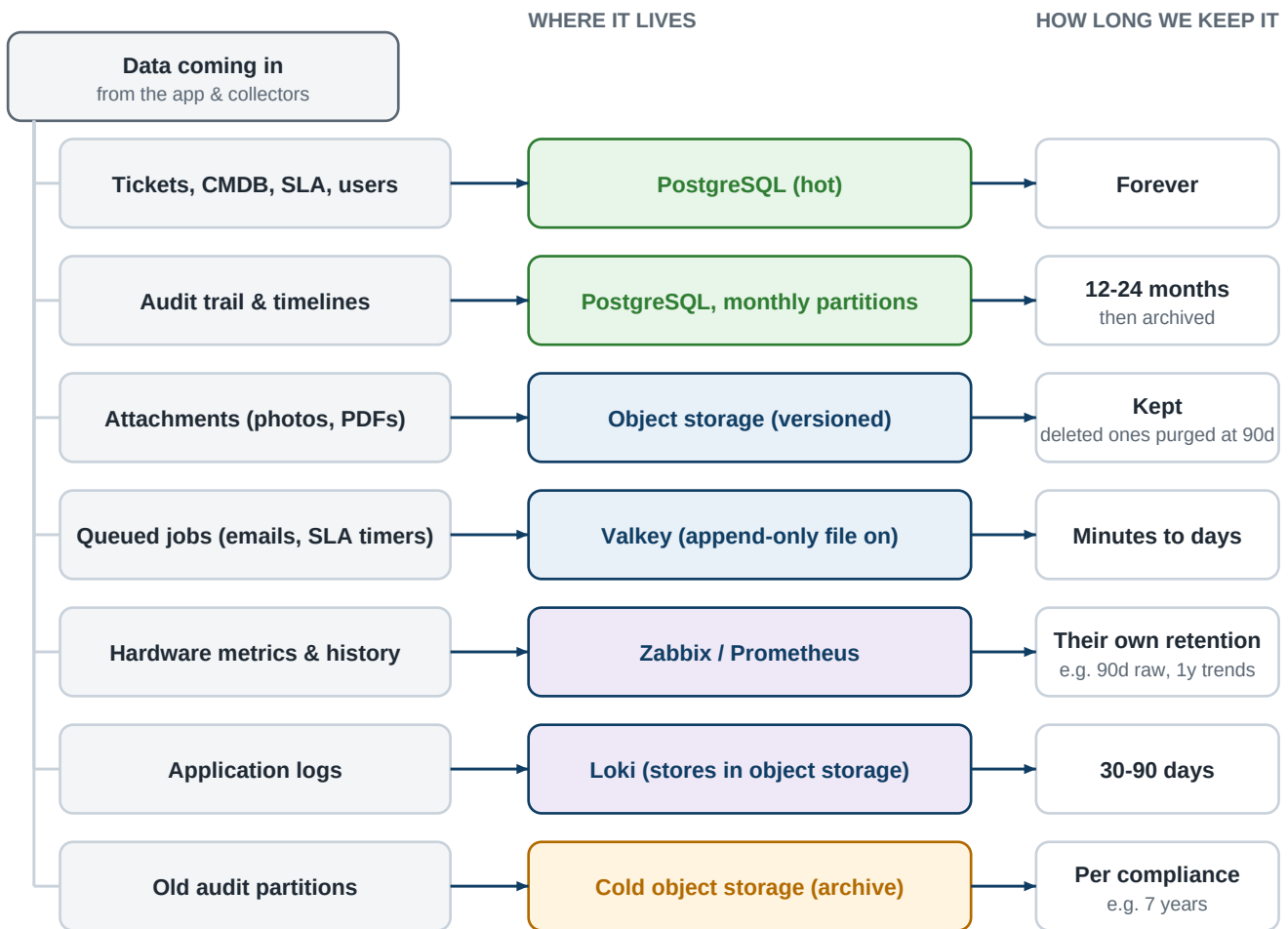
How requests and data flow once OpsDesk is live. The app tier runs on Docker Compose across two or more VMs. Green is the data that must never be lost; everything in the purple column watches the system rather than storing business data.



Two API instances mean one can restart or fail without users noticing. The standby database and Patroni do the same for the data. The site collectors only ever connect *out* to us, so iDRAC/iLO never face the internet.

Diagram 2 - Where each kind of data lives

The biggest storage mistake is putting everything in one database. Each kind of data goes to the system built for it:



Rough sizes to plan for

Data	Growth estimate (5 sites, 500 devices)	What keeps it healthy
Tickets + comments + worklogs	Small: a few GB a year	Indexes; nothing special
Audit + timeline events	Tens of millions of rows a year once telemetry is removed from it	Monthly partitions; archive after 12-24 months
Heartbeats + health polls in audit (today)	~55 million rows a year	Stop writing them to audit; they belong in Zabbix/Prometheus
Attachments	Depends on use: 25 MB max each; plan 0.5-2 TB to start	Object storage with versioning; lifecycle rules

Starting database server: 4 vCPU, 16 GB RAM, 200 GB SSD with auto-grow and an alert at 70% disk. Measure after 3 months and resize.

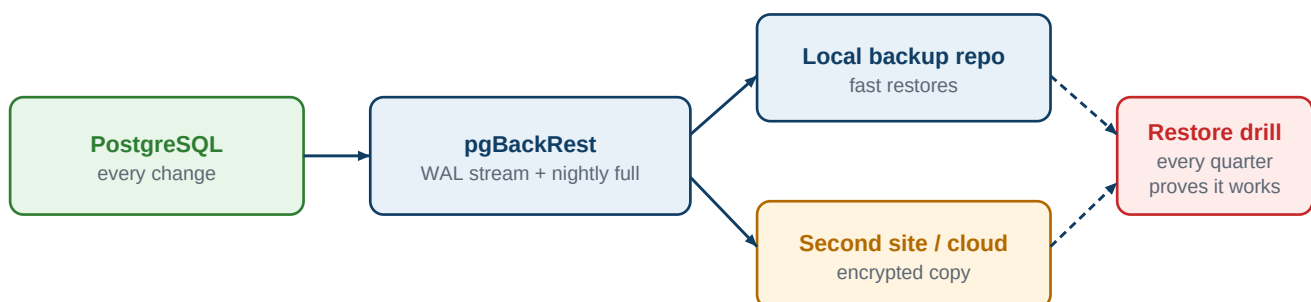
The tools - data layer

What we use to store and protect data, what it costs, and why we picked it over the alternatives.

Need	We use	Cost	Why this	Why not the others
Main database	PostgreSQL 16/17	FREE paid option	Already built on it. Handles billions of rows with partitioning. Strong transactions keep tickets, SLA and audit consistent.	MongoDB : no joins/transactions like we need for SLA + audit. MySQL : a rewrite with no gain. Managed Postgres (AWS RDS / Azure) is the paid option if we don't want to run it ourselves.
Automatic failover	Patroni + etcd	FREE	If the main database server dies, the standby takes over in seconds with no one waking up.	Manual failover means hours of downtime at 3am. Paid alternative: managed Postgres, which includes failover.
Connection pooling	PgBouncer	FREE	Lets many API instances share a small number of DB connections, so Postgres doesn't run out.	Prisma's built-in pool is per instance only; with 2+ instances and workers it multiplies.
Backups	pgBackRest	FREE	Continuous backup: restore to any minute (point-in-time), encrypted, copied offsite, fast for large databases.	pg_dump (what we have now): only nightly snapshots, slow and huge at scale, loses up to a day of tickets.
Big tables	Postgres partitioning + pg_partman	FREE	Splits audit and timeline tables by month so queries stay fast and old months can be archived cleanly.	TimescaleDB : made for metrics, which should not be in Postgres anyway; some features are under a non-open licence.
File storage	SeaweedFS (or Ceph RGW)	FREE paid option	S3-compatible, so no code change. Apache 2.0 licence, actively maintained, built for millions of files. Ceph RGW if the storage team already runs Ceph.	MinIO Community (what we use today): no longer maintained - binaries stopped Oct 2025, repo archived Apr 2026. MinIO AIStor is paid. RustFS : alpha, 'do not use in production'. Paid option: AWS S3 / Azure Blob.
Job queue	Valkey	FREE	Drop-in replacement for Redis (same protocol, BullMQ works), BSD licence, backed by the Linux Foundation, AWS and Google.	Redis 8 is AGPL - fine internally, but Valkey has the clearer licence and cloud support. Kafka/RabbitMQ : extra systems we don't need at our volume.

Diagram 3 - Backup and recovery

A backup only counts if a restore has been tested. The target: lose at most a few minutes of data, and be back within an hour.



The tools - operations and monitoring

How we see problems before users do, and how we deploy.

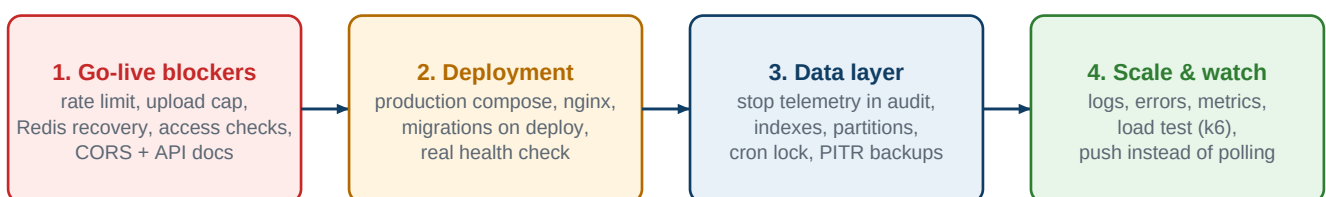
Need	We use	Cost	Why this	Why not the others
Metrics & hardware	Zabbix 7 + Prometheus	FREE	Already integrated. They store time-series properly, so Postgres keeps only current health and alerts (spec rule).	Datadog / New Relic: priced per host, expensive for a large server fleet, and data leaves our network.
Dashboards	Grafana OSS	FREE	One place for DB, API, queue and hardware graphs, with alerting to email/Teams.	Grafana Cloud is the paid hosted version - only if we don't want to run it.
Logs	Grafana Loki	FREE	Stores logs cheaply in the same object storage; small memory needs. AGPL is fine for internal use.	OpenSearch/Elasticsearch: needs 16-64 GB RAM per node and a full-text index - 3-5x more storage cost for logs.
Error tracking	GlitchTip (self-hosted)	FREE paid option	MIT licence, uses the same Sentry SDK in the code. Shows every crash with the stack trace.	Self-hosted Sentry: dozens of containers to run. Sentry SaaS: best product, paid per event - good if budget allows.
Virus scanning	ClamAV	FREE	Scans client uploads before engineers open them.	Commercial AV APIs cost per scan; ClamAV is standard for server-side scanning.
Secrets	OpenBao (or cloud secret manager)	FREE paid option	Keeps DB passwords, JWT and SMTP secrets out of .env files; rotation and audit.	HashiCorp Vault moved to a business licence; OpenBao is its open-source fork. Plain .env files leak easily.
Load balancer & TLS	nginx	FREE	Serves the web app, terminates HTTPS, spreads traffic across API instances.	Paid appliances (F5 etc.) only if the company already standardises on them.
Deployment	Docker Compose on 2+ VMs	FREE	Simple, matches the project rule (modular monolith, no Kubernetes in v1).	Kubernetes: out of scope for v1 per spec - heavy to run for one app.
Load testing	k6	FREE	Simulates hundreds of engineers and alert storms before go-live.	JMeter works too but is heavier to script.
Scheduled jobs lock	Postgres advisory locks	FREE	Built into Postgres: only one API instance runs the SLA scanner at a time. No new tool.	Redis locks add a failure mode we don't need.

Flaws to fix before go-live

Found in the current code. Items marked **blocker** would break or leak data soon after launch.

Level	Problem	What happens in production
BLOCKER	Rate limit of 100 requests/min per IP, but the incident page makes ~150/min	Engineers get 'too many requests' errors within a minute; alert webhooks get throttled in a storm.
BLOCKER	Uploads are read fully into memory before the 25 MB check	One huge upload can crash the API.
BLOCKER	A short Redis outage switches notifications off until restart	SLA emails and escalations silently stop.
BLOCKER	Alerts, vendor cases and hardware health have no role or site check	A client login can read every site's alerts.
BLOCKER	SLA scanner runs in every API instance	With 2 instances, every SLA breach is recorded twice.
BLOCKER	Deployment files don't work outside a laptop	Wrong web port, API address fixed to localhost, no migrations on deploy.
SOON	CORS open to any site; API docs public	Other websites can call the API from a user's browser.
SOON	Heartbeats and health polls written to the audit table	~55M rows/year, slower audit search every month.
SOON	Missing indexes; Command Center loads every incident ever	Dashboards and audit search slow down, then time out.
SOON	Health check always says OK; no clean shutdown	Traffic keeps going to a broken instance; restarts cut requests.
SOON	No error tracking, metrics or structured logs	We find out about outages from users.
SOON	MinIO Community no longer maintained	No security fixes for the attachment store.

Diagram 4 - The order to do it in



Free vs paid - the decision in one table

Everything can run for free. Pay only where it saves more staff time than it costs.

Area	Free path (self-hosted)	Paid path (someone runs it for us)	Recommendation
Database	PostgreSQL + Patroni + pgBackRest + PgBouncer	AWS RDS / Azure Database for PostgreSQL	Free if the team has a DBA; otherwise managed
Files	SeaweedFS or Ceph RGW	AWS S3 / Azure Blob / MinIO AIStor	Free on-premises for a data-center company
Queue	Valkey with Sentinel	Managed Valkey/Redis (ElastiCache, Azure Cache)	Free
Monitoring	Zabbix, Prometheus, Grafana, Loki	Grafana Cloud / Datadog	Free (already integrated)
Errors	GlitchTip	Sentry SaaS	Free to start; Sentry if budget allows
Support contracts	Community forums	Zabbix, Percona/EDB (Postgres) support	Consider for the database once live

What we deliberately are not using

- **Kubernetes** - out of scope for v1 per the build spec; Docker Compose on two or more VMs is enough.
- **Microservices** - the spec requires a modular monolith; splitting adds network failures without adding capacity we need.
- **A time-series database inside Postgres** - metrics belong in Zabbix/Prometheus (spec rule: keep telemetry out of Postgres).
- **MinIO Community, RustFS** - one is unmaintained, the other is alpha software.
- **Per-host SaaS monitoring** (Datadog, New Relic) - cost grows with every server we manage, and data leaves our network.

Sources for licence and status claims

MinIO CE status: glukhov.org/data-infrastructure/object-storage/minio-dead - itsfoss.com/news/minio-moves-away-from-open-source. Redis/Valkey licences: theregister.com/2025/08/14/redis_redux - oneuptime.com (Redis vs Valkey, Mar 2026). Zabbix AGPL: blog.zabbix.com (Zabbix 7.0 AGPLv3). Loki AGPL: grafana.com/docs/loki/latest/community/faqs. Sentry FSL / GlitchTip MIT: bugsink.com/blog/glitchtip-vs-sentry-vs-bugsink. SeaweedFS/Ceph/Garage: blog.eleest.io (MinIO alternatives). Checked September 2026; licences change - re-check before signing anything.